

Tema 6

Búsqueda entre Adversarios

Razonamiento y Planificación Automática

UNIR · Maestría en Inteligencia Artificial · 2026

1. Introducción: el problema con adversarios

En los temas anteriores el agente operaba en un entorno **pasivo**: buscaba el camino de menor coste hacia una meta sin que nadie interfiriera. Ahora el entorno tiene **múltiples agentes**, cada uno con sus propios objetivos, donde las acciones de uno afectan directamente al rendimiento del otro.

Problema entre adversarios

Escenario donde más de un agente especializado actúa de modo concurrente en un mismo entorno. Cada agente es independiente, intenta conseguir sus propios objetivos, y puede intentar *predecir* el comportamiento de los demás, aunque no puede controlarlo.

1.1 Tipos de escenarios multiagente

Tipo	Descripción	Ejemplo
Cooperativo	Todos los agentes comparten la misma meta. Las acciones de unos facilitan los objetivos de todos.	Robots coordinados para mover carga pesada
Parcialmente cooperativo	Algunas metas son compartidas, otras son opuestas.	Dos empresas con interés común en el mercado

Antagónico (suma cero)	Las metas son completamente opuestas. Todo lo que gana uno lo pierde el otro.	Ajedrez, damas, tres en raya
-----------------------------------	--	------------------------------

Origen del término "suma cero": proviene de la Teoría de Juegos (von Neumann, 1928). Si se suman las ganancias de todos los jugadores, el resultado siempre es cero: no se crea ni se destruye valor, solo se transfiere entre jugadores.

1.2 Clasificación de los juegos

Los juegos de suma cero se clasifican en tres dimensiones independientes:

Dimensión	Opción A	Opción B
Número de jugadores	Bipersonal (ajedrez, damas)	Multijugador (Monopoly, Catán)
Elementos de azar	Determinista (ajedrez)	Estocástico (backgammon, parchís)
Información	Perfecta (damas — tablero visible)	Imperfecta (blackjack — cartas ocultas)

1.3 Representación formal: el árbol de juego G

Árbol de juego G

Formalmente definido como la tupla $G = \langle N, E, L \rangle$, donde:

- N : conjunto de nodos (cada nodo = estado del juego, una "fotografía" del tablero)
- $E \subseteq N \times N$: función que asigna a cada nodo sus hijos (estados sucesores)
- $L = \{max, min\}$: función que etiqueta cada nodo indicando a quién le toca el turno

Los dos jugadores se denominan por convención **máx** y **min**:

- **Máx**: quiere *maximizar* la utilidad. Empieza siempre. Usa fichas X.
- **Mín**: quiere *minimizar* la utilidad de máx. Usa fichas O.

Cada nivel del árbol representa **media jugada**, denominada *ply*. Un turno completo (máx juega + min responde) equivale a dos plys.

1.4 Función de utilidad U(s)

La función de utilidad se define inicialmente solo en los **nodos terminales**:

Resultado	U(s)
Gana máx	$+\infty$
Empate	0

Gana min	$-\infty$
----------	-----------

El algoritmo debe **propagar** estos valores desde las hojas hasta la raíz para que máx pueda decidir qué movimiento hacer desde el estado inicial.

1.5 Conocimiento a priori del agente

Cada agente dispone del siguiente conocimiento mínimo:

- s_0 : posición inicial (estado inicial)
- $expandir : s \mapsto \{s_{i_1}, \dots, s_{i_n}\}$: conjunto finito de posiciones sucesoras
- $terminal? : s \mapsto \text{true} \mid \text{false}$: prueba terminal
- $U : s \mapsto k, k \in \mathfrak{R}$: función parcial de utilidad del juego

2. Algoritmo Minimax

Minimax

Método de decisión para *minimizar la pérdida máxima esperada* en juegos con adversario y con información perfecta. Estrategia óptima para máx: **maximizar la utilidad mínima garantizada en cada jugada**, asumiendo que min siempre jugará de la manera más perjudicial posible para máx.

2.1 Intuición

Máx tiene tres movimientos posibles. Para cada uno, min responderá con la peor jugada posible para máx. Entonces máx razona:

Lógica de máx:

"Si elijo a1, min me garantiza como mínimo X. Si elijo a2, min me garantiza como mínimo Y. Si elijo a3, min me garantiza como mínimo Z. Elijo el movimiento con el mayor mínimo garantizado."

2.2 Mecánica: propagación de valores

El algoritmo trabaja en tres pasos:

1. **Generar el árbol completo** hasta los nodos terminales.
2. **Aplicar $U(s)$** a cada nodo terminal.
3. **Propagar hacia arriba** siguiendo la regla:
 - Nodo **máx** → toma el *máximo* de sus hijos
 - Nodo **min** → toma el *mínimo* de sus hijos

2.3 Ejemplo numérico

Árbol con 3 movimientos para máx y 3 hojas por movimiento:

Hojas: $a1 \rightarrow \{3, 12, 8\}$, $a2 \rightarrow \{2, 4, 6\}$, $a3 \rightarrow \{14, 5, 2\}$

Nivel min:

- $\min(3, 12, 8) = 3$ para a1
- $\min(2, 4, 6) = 2$ para a2
- $\min(14, 5, 2) = 2$ para a3

Nivel máx (raíz):

$$\max(3, 2, 2) = 3 \rightarrow \text{jugada óptima: a1}$$

Máx elige a1 porque, aunque min juegue perfecto, lo peor que puede pasarle es obtener 3. Los otros caminos solo garantizan 2.

2.4 Pseudocódigo

El algoritmo usa dos funciones **mutuamente recursivas**. α arranca en $-\infty$

(máx sube desde el piso) y β arranca en $+\infty$ (min baja desde el techo):

```
Función MaxValor(estado)
  Si terminal?(estado)
    devolver U(estado)
  sucesores  $\leftarrow$  expandir(máx,
estado)
```

```
Función MinValor(estado)
  Si terminal?(estado)
    devolver U(estado)
  sucesores  $\leftarrow$  expandir(min,
estado)
```

$$\alpha \leftarrow -\infty$$

Para cada $s \in \text{sucesores}$:

$$\alpha \leftarrow \max(\alpha, \text{MinValor}(s))$$

$\text{MinValor}(s)$

devolver α

$$\beta \leftarrow +\infty$$

Para cada $s \in \text{sucesores}$:

$$\beta \leftarrow \min(\beta, \text{MaxValor}(s))$$

$\text{MaxValor}(s)$

devolver β

2.5 Complejidad y el problema de escala

Complejidad de Minimax

$$O(b^d)$$

donde b = factor de ramificación (movimientos posibles por turno) y d = profundidad del árbol (número de plys hasta los nodos terminales).

Para el ajedrez: $b \approx 35$, $d \approx 100 \rightarrow 35^{100}$ nodos.

Computacionalmente imposible.

2.6 Minimax con suspensión

La solución práctica: en lugar de expandir hasta los nodos terminales reales, **suspender la expansión a una profundidad d fija** y evaluar esos nodos con una función heurística.

Función de evaluación heurística $e(s)$

Estima la utilidad esperada del juego para una posición s no terminal. Requisito

fundamental: e debe coincidir con U en los nodos terminales reales.

Forma más habitual — función lineal ponderada:

$$e(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

donde cada $f_i(s)$ es una característica del estado y w_i su peso relativo.

Ejemplos de funciones de evaluación:

Ajedrez: $e(s) =$ suma de los valores materiales en s

(reina=9, torre=5, alfil=3, caballo=3, peón=1)

Tres en raya: $e(s) =$ líneas abiertas para máx en s – líneas abiertas para

min en s

Consecuencia importante: con heurísticas fuertes se pierde la garantía de optimalidad. Se sacrifica optimalidad por tratabilidad computacional. La calidad del resultado depende directamente de la calidad de la heurística.

3. Poda Alfa-Beta

Poda Alfa-Beta ($\alpha - \beta$)

Optimización de minimax que descarta ramas del árbol que nunca van a influir en la decisión final, sin alterar el resultado. Permite tomar la misma decisión minimax correcta **sin evaluar todos los nodos**.

3.1 Intuición

Si máx ya tiene garantizado un valor de 3 por el camino a_1 , y al explorar a_2 el primer hijo devuelve 2, min nunca va a dejar que a_2 supere 2. Como $2 < 3$, no importa qué valores tengan los otros hijos de a_2 — ese subárbol es **irrelevante** para la decisión final.

Principio clave: la poda no corta porque conoce el mínimo exacto de la rama — corta porque ya sabe que ese mínimo *no puede superar el umbral que le interesa a máx* . El resultado es idéntico al minimax sin poda.

3.2 Los parámetros α y β

El algoritmo mantiene dos valores que **viajan juntos** por todo el árbol como parámetros de las llamadas recursivas:

Parámetro	Significado	Valor inicial
α	Mejor valor que máx ha encontrado hasta el momento en el camino desde la raíz. El "piso" de máx — lo mínimo ya garantizado.	$-\infty$
β	Mejor valor que min ha encontrado hasta el momento en el camino desde la raíz. El "techo" de min — lo máximo que min está dispuesto a dejar pasar.	$+\infty$

El intervalo $[\alpha, \beta]$ se va **cerrando** a medida que se explora el árbol.

3.3 Las dos condiciones de poda

Poda alfa

(ocurre en nodo min)

Condición: $\beta \leq \alpha$

Min está evaluando sus hijos y

encuentra un valor $v \leq \alpha$. Min

va a devolver algo $\leq v \leq \alpha$.

Pero máx ya tiene α garantizado

— ese nodo min nunca mejorará la situación de máx.

→ Se descartan los hijos restantes de ese nodo min.

Poda beta

(ocurre en nodo máx)

Condición: $\alpha \geq \beta$

Máx está evaluando sus hijos y

encuentra un valor $v \geq \beta$. Máx

va a devolver algo $\geq v \geq \beta$.

Pero min ya tiene β garantizado

— min nunca va a permitir que ese nodo máx sea elegido.

→ Se descartan los hijos restantes de ese nodo máx.

Las dos podas son **simétricas y duales**. La poda alfa aplica desde la perspectiva de máx (cuando min trabaja), la poda beta desde la perspectiva de min (cuando máx trabaja).

3.4 ¿Cuándo NO se poda?

Min explora los hijos de un nodo y **ninguno** es $\leq \alpha$. En ese caso, min devuelve el

mínimo de todos sus hijos exactamente igual que en minimax normal. La poda alfa-beta es **oportunista**: solo actúa cuando hay evidencia de irrelevancia.

Ejemplo con $\alpha=1$ en a2 (sin poda):

Hijos de a2: {4, 3, 2}. Ninguno es ≤ 1 . Min devuelve 2 a máx. Máx actualiza:

$$\alpha = \max(1, 2) = 2$$

Resultado: máx ahora tiene un piso más alto. Se evalúan todos los hijos sin corte.

3.5 Pseudocódigo con poda

Función MaxValor(estado, α ,

β)

Si suspensión?(estado)

devolver e(estado)

sucesores $\leftarrow$ expandir(máx,
estado)

Para cada s $\in$ sucesores:

$\alpha \leftarrow \max(\alpha,$

MinValor(s, α , β))

Si $\alpha \geq \beta \rightarrow$ **devolver**

α

devolver α

Función MinValor(estado, α ,

β)

Si suspensión?(estado)

devolver e(estado)

sucesores $\leftarrow$ expandir(min,
estado)

Para cada s $\in$ sucesores:

$\beta \leftarrow \min(\beta,$

MaxValor(s, α , β))

Si $\beta \leq \alpha \rightarrow$ **devolver**

β

devolver β

3.6 Ejemplo numérico paso a paso

Mismo árbol: hojas $a1=\{3,12,8\}$, $a2=\{2,4,6\}$, $a3=\{14,5,2\}$

1. Evaluando a1:

$\min(3,12,8)=3$. La raíz actualiza $\alpha = 3$.

2. Evaluando a2:

primer hijo = 2. Min tiene $\beta = 2$. Como $\beta(2) \leq \alpha(3) \rightarrow$

PODA. Se descartan los hijos 4 y 6. Ya sabemos que a2 devuelve ≤ 2 , que es peor que 3.

3. Evaluando a3:

hijos 14, 5, 2. β va a 14, luego a 5, luego a 2. Con

$\beta = 2 \leq \alpha = 3 \rightarrow$ **PODA** tras el tercer hijo (aquí no ahorra, pero en

árboles más profundos sí).

4. Resultado:

$\max(3, \leq 2, 2) = 3 \rightarrow$ jugada óptima a1. Idéntico a minimax sin poda.

3.7 Análisis de complejidad

Algoritmo	Complejidad	Con $b=10, d=4$
Minimax sin poda	$O(b^d)$	$10^4 = 10.000$ nodos
Minimax con poda (caso medio)	$O(b^{3d/4})$	$10^3 = 1.000$ nodos

Minimax con poda (mejor caso)	$O(b^{d/2})$	$10^2 = 100$ nodos
-------------------------------	--------------	--------------------

El mejor caso ocurre cuando los nodos están ordenados perfectamente — es decir, cuando siempre se explora primero el mejor movimiento. En ese caso la poda divide el exponente a la mitad, equivalente a poder buscar el **doblo de profundidad** con el mismo costo computacional.

La eficiencia de minimax con poda $\alpha - \beta$ depende del orden en que se

exploran los nodos. Con orden aleatorio se acerca al caso medio; con mal orden, al peor caso (sin poda efectiva).

Punto crítico para el examen: la poda alfa-beta siempre produce el **mismo resultado** que minimax sin poda. No es una aproximación — es una optimización exacta. Lo único que cambia es la eficiencia, nunca la decisión.

4. Expectiminimax: juegos con azar

Minimax y poda alfa-beta asumen que el juego es **determinista**. Cuando el entorno introduce aleatoriedad (dados, monedas, cartas), el modelo debe extenderse.

Expectiminimax

Extensión de minimax para juegos bipersonales con elementos de azar. Añade un tercer tipo de nodo al árbol: el **nodo azar**, que representa los momentos donde ocurre un evento aleatorio independiente de los jugadores.

4.1 El nodo azar

Los hijos de un nodo azar son todos los posibles resultados del evento aleatorio (6 caras de un dado, cara/cruz de una moneda), y cada hijo tiene asociada una **probabilidad de ocurrencia**

$$p(s) \quad \text{con} \quad \sum p(s) = 1 \quad .$$

El árbol ya no alterna solo entre niveles máx y min — ahora puede tener niveles **azar** intercalados.

4.2 Propagación del valor en nodo azar

En un nodo azar no se toma el máximo ni el mínimo — se calcula el **valor esperado**:

Fórmula completa de ExpectMinimax(n):

$$e(n)$$

si n es nodo
suspensión

$$\max_{s \in \text{expandir}(n)} \text{ExpectMinimax}(s)$$

si n es nodo máx

$$\min_{s \in \text{expandir}(n)} \text{ExpectMinimax}(s)$$

si n es nodo min

$$\sum_{s \in \text{expandir}(n)} p(s) \cdot \text{ExpectMinimax}(s)$$

si n es nodo azar

4.3 Ejemplo numérico: Backgammon simplificado

Se lanza una moneda: cara (prob. 1/2, valor 1), cruz (prob. 1/2, valor 2).

Nodo azar con dos hijos:

- Si sale cara → estado con evaluación
10
- Si sale cruz → estado con evaluación
9.5

Valor del nodo azar:

$$\frac{1}{2} \cdot 10 + \frac{1}{2} \cdot 9,5 = 5 + 4,75 = 9,75$$

Máx no puede elegir el 10 directamente — debe "promediar" porque el dado puede no favorecerle.

4.4 Diferencia crítica: la escala importa

En minimax puro: multiplicar todos los valores de utilidad por 10 no cambia la decisión. Las relaciones de orden se preservan.

En expectiminimax: la escala SÍ afecta la decisión. El valor esperado es una suma ponderada — distorsionar la escala distorsiona los promedios y puede cambiar qué rama parece mejor.

Por eso hay dos reglas para la función de evaluación en expectiminimax:

1.
No usar $+\infty$ ni $-\infty$ en nodos de suspensión — esos valores se reservan para victorias/derrotas reales. Usar un **intervalo finito** (ej. entre 0 y 1).
2.
La función e debe ser idealmente una **transformación lineal positiva** de la probabilidad de ganar: $e(s)$ debe crecer proporcionalmente con la probabilidad real de victoria.

Función de evaluación para Backgammon:

$$e(a, b, c, d) = a + b + c + d$$

donde a, b, c, d son las posiciones de las fichas. Cuanto más cerca del 5 (meta), mejor para máx.

Para el estado (3,4,1,2): $e = 3 + 4 + 1 + 2 = 10$

4.5 Complejidad de expectiminimax

Complejidad de Expectiminimax

$$O(b^d \cdot n^d)$$

donde b = factor de ramificación, d = profundidad en plys, n = número de posibles resultados del evento de azar.

Para backgammon con 2 dados: $n = 21$ combinaciones, $b \approx 20$

movimientos:

Plys anticipados (d)	Nodos en el árbol
1	$20 \times 21 = 420$
2	176.400

3	74.088.000
---	------------

El crecimiento es brutal. Por eso en juegos con azar es especialmente importante usar suspensión a profundidades bajas y funciones de evaluación bien calibradas. Es preferible aplicar expectiminimax en el **máximo número de plys posibles** y solo al final aplicar la función e .

5. Cuadro comparativo general

Situación	Algoritmo	Complejidad	Optimalidad
Juego determinista, árbol pequeño	Minimax básico	$O(b^d)$	Óptimo
Juego determinista, árbol grande	Minimax con suspensión	$O(b^d)$ hasta profundidad d	Depende de e(s)
Juego determinista, optimizado	Minimax con poda $\alpha - \beta$	$O(b^{d/2})$ mejor caso	Óptimo (igual que sin poda)
Juego con azar	Expectiminimax	$O(b^d \cdot n^d)$	Depende de e(s)

6. Dudas frecuentes y confusiones comunes

1. ¿Por qué min no poda la rama completa de a2 desde el inicio?

Porque min no sabe de antemano cuál va a ser el mínimo de sus hijos sin explorarlos. Si el primer hijo de a2 vale 2 y $\alpha=1$, min *no puede asumir* que los demás hijos serán menores. Alguno podría ser 5 o 10 — un valor mayor que α que sí le interesaría a máx. La poda solo ocurre cuando min ya tiene evidencia de que su resultado final será $\leq \alpha$.

2. ¿Qué pasa si min no encuentra ningún valor $\leq \alpha$?

Simplemente devuelve el mínimo de todos sus hijos, exactamente igual que en minimax normal. La poda alfa-beta es oportunista — si no hay oportunidad de podar, el algoritmo se comporta exactamente como minimax.

3. ¿La poda alfa-beta puede dar un resultado diferente al minimax?

No. Siempre produce el **mismo resultado**

. Es una optimización exacta, no una aproximación. Solo cambia la eficiencia (menos nodos evaluados), nunca la decisión final.

4. ¿Por qué la escala importa en expectiminimax pero no en minimax?

En minimax solo importan las relaciones de orden (mayor/menor). En expectiminimax se calculan sumas ponderadas (valor esperado), y las sumas son sensibles a la escala absoluta de los valores. Una transformación no lineal puede cambiar cuál rama tiene mayor valor esperado.

5. ¿Qué es un ply?

Media jugada — el movimiento de un solo jugador. Un turno completo (máx juega + min responde) equivale a dos plys. La profundidad de suspensión d se mide en plys, no en turnos completos.

7. Claves para la evaluación

- La **estrategia óptima** es aquella que implica el mejor resultado garantizado para **máx**.
- Minimax genera el árbol completo **en profundidad primero** hasta los nodos de suspensión.
- La poda $\alpha - \beta$ **siempre produce el mismo resultado** que sin poda —

nunca altera la decisión.
- La eficiencia de la poda **depende del orden de exploración** de los nodos.
- Expectiminimax añade el azar como un **nuevo jugador**, siempre que haya un evento con independencia de los jugadores cuyo resultado sea aleatorio.
- El caso ideal de la función e en expectiminimax es una **transformación lineal positiva** de la probabilidad de ganar.
- La complejidad de expectiminimax es **proporcional al número de nodos en el árbol**: $O(b^d \cdot n^d)$.
- Juegos: clasificación en número de jugadores, elementos de azar e información — las **tres dimensiones** siempre van juntas.

8. Respuestas al test oficial

Pregunta	Respuesta	Justificación
----------	-----------	---------------

1. Estrategia óptima	A	Implica el mejor resultado garantizado para máx.
2. Tipos de problemas multiagente	C	Cooperativos + parcialmente cooperativos + antagónicos.
3. Clasificación de juegos	A	Número de jugadores, elementos de azar e información.
4. Estrategia óptima para máx	B	Maximizar la utilidad mínima en cada jugada — minimax.
5. Minimax genera	B	Árbol completo, primero en profundidad, hasta nodos de suspensión.
6. Poda alfa-beta	A	Produce siempre el mismo resultado que sin poda.
7. Expectiminimax	C	Añade azar como jugador, solo cuando hay evento aleatorio independiente.
8. Función e ideal en expectiminimax	A	Transformación lineal positiva de la probabilidad de ganar.
9. Complejidad de expectiminimax	B	Proporcional al número de nodos en el árbol.
10. Eficiencia poda alfa-beta	C	Depende del orden en que se exploran los nodos.